

Software Architecture and Design Specification

Project: Vehicle Parking System (VPS)

Version: 1.0 | **Date:** 09-09-2026 | **Status:** Final Document

Authors: Himansh Ahuja (PES2UG24CS923), Shweta Mailaragouda Patil (PES2UG25CS825), Angelo Arakal (PES2UG24CS062), Akarsh Parashuram (PES2UG24CS040)

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Vehicle Parking System (VPS).

1.2 Scope

- Covers vehicle entry/exit automation at gate lanes utilizing Automatic Number Plate Recognition (ANPR) and QR tickets.
- Includes integration with payment gateways for cashless settlement, real-time slot vacancy detection via IoT sensors, and multi-category parking allocation.
- Encompasses slot pre-booking, operational monitoring, and reporting.
- Explicitly excludes internal payment switch banking implementations and physical towing enforcement.

1.3 Audience

Developers, QA Engineers, System Integrators, Maintenance Technicians, and Assessment Evaluators.

1.4 Definitions

Acronyms used include VPS, ANPR, RFID, QR, POS, UI, API, TLS, PCI-DSS, RTO, RPO, EV, and VMS.

2. Document Overview

2.1 How to use this document

This document provides architectural deliverables, including UML sequence diagrams, component details, threat models, and API design specifications.

2.2 Related Documents

- SRS_Vehicle_Parking_System.pdf

3. Architecture

3.1 Goals & Constraints

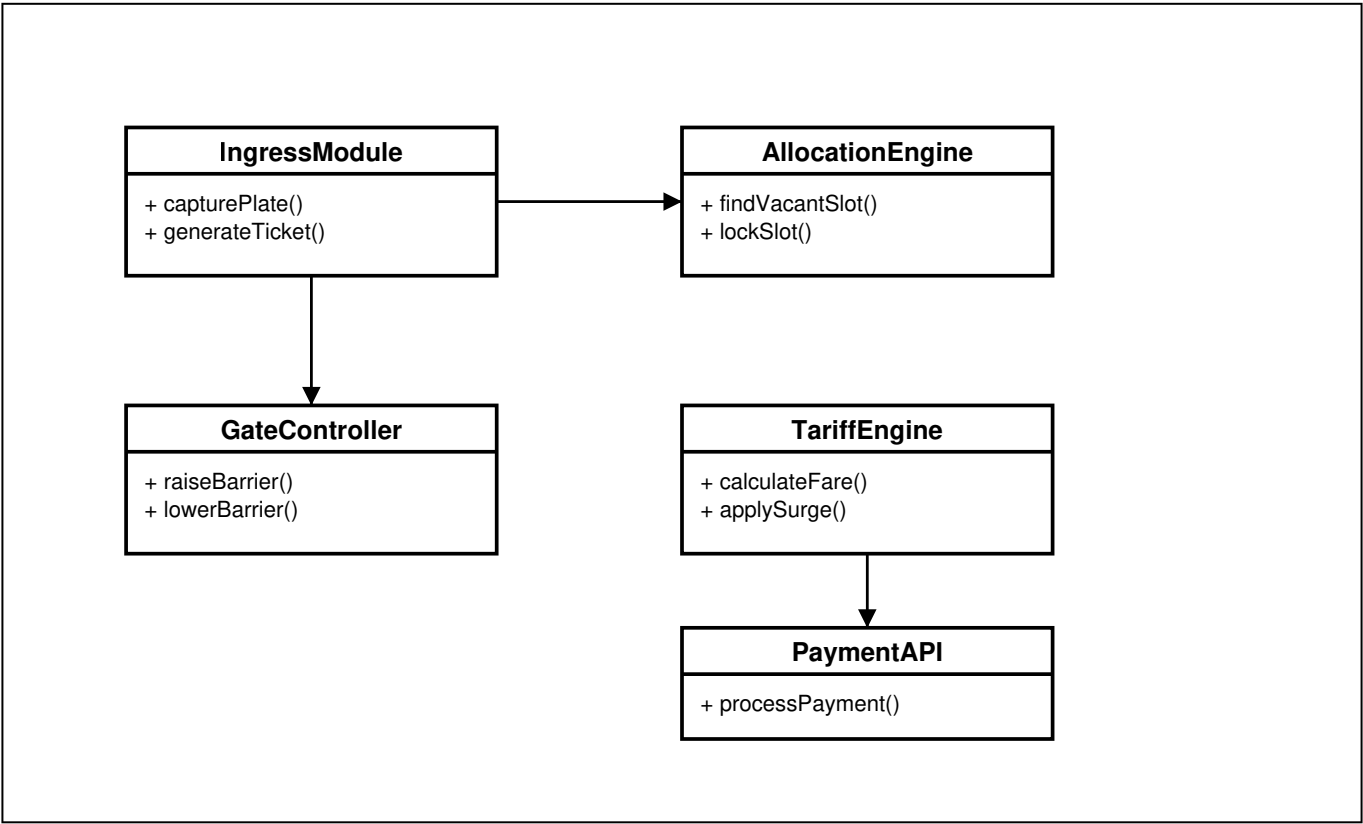
- **Goals:** Secure, reliable, 99.9% monthly availability, and overall vehicle ingress latency of ≤ 3 seconds for 90% of transactions.
- **Constraints:** PCI-DSS payment security compliance, cryptographic data encryption at rest and in transit, and hardware protocol limitations (Wiegand/RS-485).

3.2 Stakeholders & Concerns

- **Customer/Driver:** Expects entry/exit in under 3 seconds, intuitive navigation, and fast mobile payments.
- **Parking Attendant:** Needs to manage exceptions, unreadable plates, cash billing, and manual barrier overrides via a fast-POS screen.
- **System Administrator:** Requires configuration capabilities for tariffs, user permissions, and system health monitoring.
- **Facility Manager:** Reviews revenue reconciliation reports, peak occupancy trends, and security audit logs.

3.3 Component (UML) Diagram

The following UML Class/Component diagram illustrates the structural relationships between the core processing units of the Vehicle Parking System.



Component Descriptions

- **Ingress Module:** Interfaces with ANPR cameras to capture plates and generate QR-coded tickets via thermal printers.
- **Allocation Engine:** Designates the nearest vacant slot based on vehicle category (2W, 4W, EV, Handicap).
- **Sensor Hub & Signage Service:** Gathers telemetry from ultrasonic slot sensors to toggle LED statuses (Red/ Green) and updates VMS display boards.
- **Tariff Engine:** Computes dynamic parking fees based on duration, vehicle category, tiered rates, 15-minute grace periods, and surge pricing.
- **Payment Gateway API:** Facilitates cashless transactions via dynamic UPI QR, Credit/Debit cards, and wallets.

3.4 Chosen Architecture Pattern and Rationale

An edge-to-cloud IoT architecture is chosen. Edge gate controllers provide autonomous offline transaction queuing for up to 72 hours during wide-area network failures, auto-syncing upon reconnection.

3.5 Technology Stack & Data Stores

- **Hardware:** Motorized boom barriers, ANPR IP cameras, 2D QR thermal printers, ultrasonic/magnetic slot sensors, and payment POS terminals.
- **Software OS & Data:** Embedded gate controller OS running an SQLite offline store-and-forward queue. The centralized backend utilizes a PostgreSQL database and a Redis memory store.
- **Network:** TLS 1.3 enforced for API channels, MQTT/WebSocket for live slot telemetry.

3.6 Risks & Mitigations

- **Risk:** Network failure causing system paralysis.
Mitigation: Edge gate controllers utilize an SQLite offline queue allowing lane functions to continue for up to 72 hours.
- **Risk:** Counterfeit ticket fraud.
Mitigation: Entry ticket QR codes are digitally signed with HMAC-SHA256.

3.7 Traceability to Requirements

- VPS-F-001 (Plate Capture) → Ingress Module
- VPS-F-005 (Slot Assignment) → Allocation Engine
- VPS-F-008 (Dynamic Fare) → Tariff Engine

3.8 Security Architecture

- **Data Protection:** All sensitive PII data must comply with PCI-DSS.
- **Encryption:** Passwords hashed with Argon2id/bcrypt and sensitive records at rest encrypted using AES-256.
- **Access Control:** Enforcement of Role-Based Access Control (RBAC) and Multi-Factor Authentication (MFA) via TOTP.
- **Auditing:** Maintenance of immutable, write-once audit logs for all barrier overrides and tariff edits, retained for 3 years.

4. Design

4.1 Design Overview

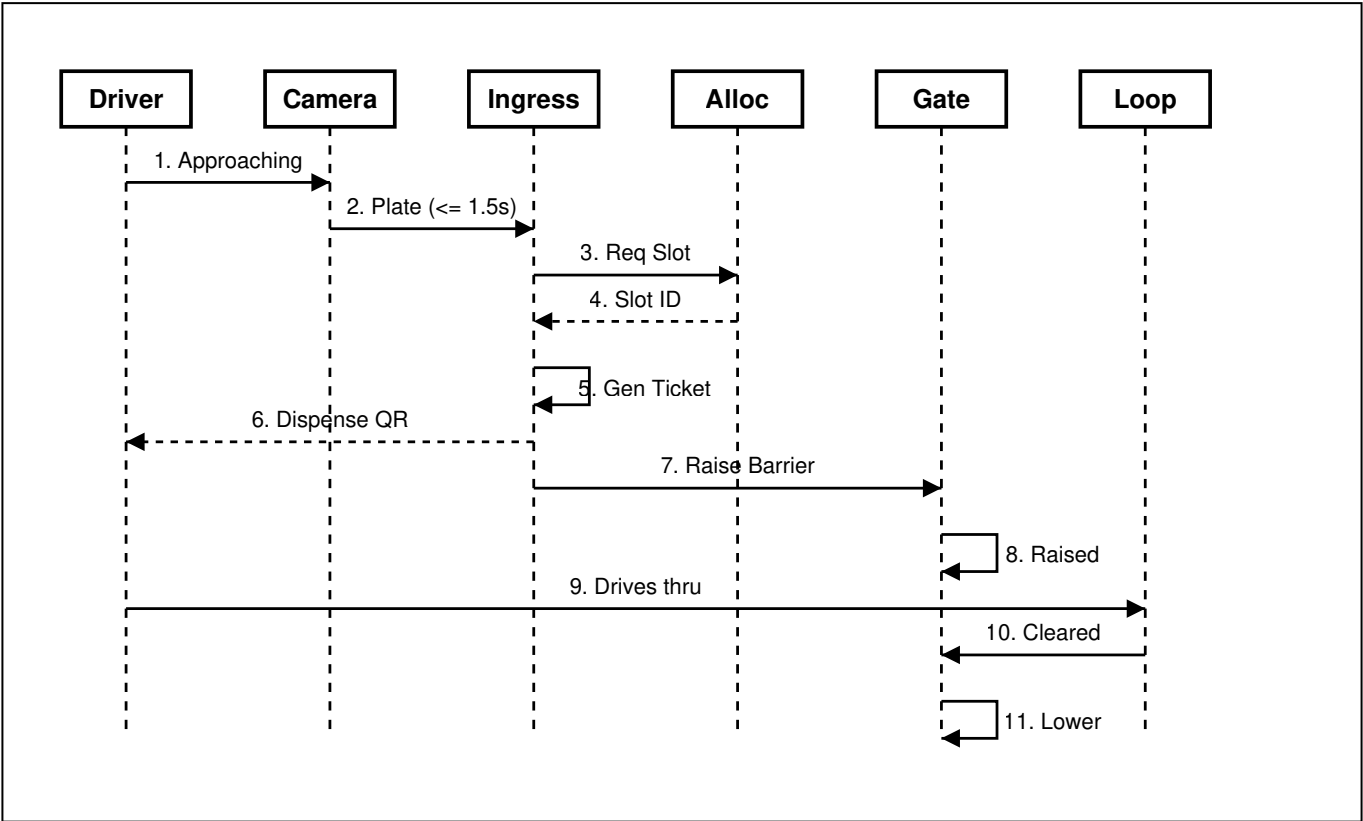
The system is modularized to separate physical gate actuation (GateController), IoT sensor data streaming (SensorHub), and transactional billing (TariffEngine), ensuring that hardware latency does not bottleneck the payment and analytics processing.

4.2 UML Sequence Diagrams

The sequence diagrams below detail the interactive workflows for vehicle entry and payment-verified exit.

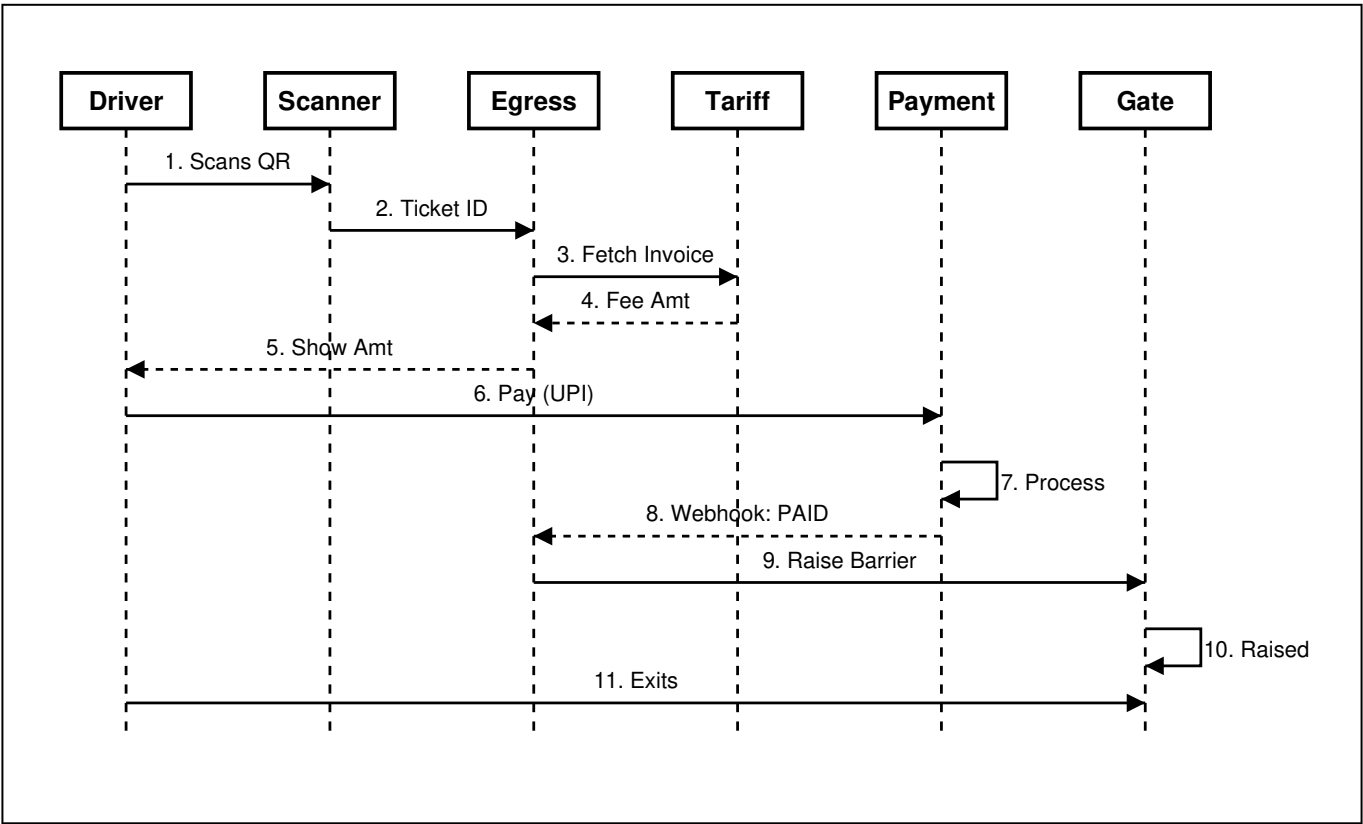
Flow 1: Vehicle Ingress Flow

Customer approaches → ANPR reads plate → System generates unique encrypted QR ticket → System triggers relay to raise ingress boom barrier → Ground loop safety check.



Flow 2: Cashless Egress Flow

Driver scans QR at exit → Egress Module fetches invoice → Payment webhook processes transaction via API → GateController raises egress barrier upon "PAID" status receipt.



4.3 API Design

Endpoint: /api/v1/ingress/ticket

- **Method:** POST
- **Request:** { "plate_number": "string", "vehicle_category": "string", "timestamp": "ISO8601" }
- **Response:** { "ticket_id": "uuid", "qr_payload": "hmac-sha256-string", "allocated_slot": "string" }
- **Errors:** 400 Invalid Plate, 503 Edge Controller Offline

Endpoint: /api/v1/egress/checkout

- **Method:** POST
- **Request:** { "ticket_id": "uuid", "payment_method": "upi/card" }
- **Response:** { "status": "PAID", "receipt_url": "string", "barrier_action": "OPEN" }

4.4 Error Handling, Logging & Monitoring

Hardware errors are routed to the Parking Attendant fast-POS screen for manual intervention. Diagnostic telemetry and audit logs are securely stored.

4.5 UX Design

The user interface utilizes a sunlight-readable kiosk touchscreen equipped with multilingual audio-visual prompts to assist drivers during entry and payment.

5. Appendices

5.1 Glossary

- **ANPR:** Automatic Number Plate Recognition
- **PCI-DSS:** Payment Card Industry Data Security Standard
- **VMS:** Variable Message Signs
- **EV:** Electric Vehicle

5.2 References

- SRS_Vehicle_Parking_System.pdf (Version 1.0, 09-09-2026)